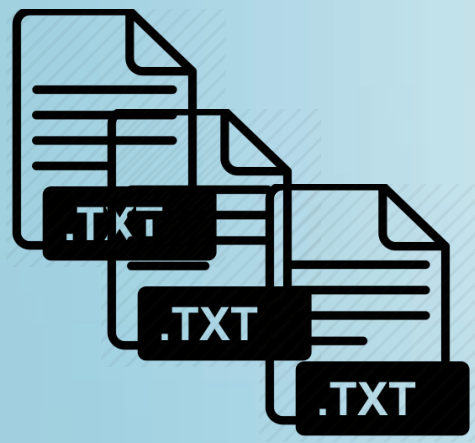
PYTHON PROGRAMMING COURSE

OUTLINE

- Files
- File manipulation
- Reading from files
- Standard file object
- Writing to files
- Built-in methods
- Modifying files and directories
- Exceptions
- Handling exceptions
- Raising exceptions
- Creating exceptions

FILES

- 2 categories
  - text : sequence of line terminated with special character called EOL
  - binary



FILE MANIPULATION

- In Python, to manipulate files, we have to create file object
- There are two ways to create file objects
- open() method
  - First argument is filename, second is mode
  - Modes = ‘r’ for reading (default), ‘w’ for writing, ‘a’ for appending, ‘r+’ for reading and writing, ‘b’ for binary mode

```
>>> f = open(”notes.txt", 'rb')
```

- file() constructor
  - Depreciate in Python 3

```
>>> f = file(”notes.txt", 'r')
```

- Note: when a file operation fails, an `IOError` exception is raised.

READING FROM FILES

- f.read()
  - Returns the entire contents of a file as a string
  - Provide an argument to limit the number of characters you pick up
- f.readline()
  - Returns each line of a file as a string (ends with a newline).
  - End-of-file reached when return string is empty
- f.readlines()
- Use for loop to repeatedly read file contents

```
>>> f = open("somefile.txt", 'r')
>>> f.read()
"Here's a line.\nHere's another line.\n"
>>> f.close()
>>> f = open("somefile.txt", 'r')
>>> f.readline()
"Here's a line.\n"
>>> f.readline()
"Here's another line.\n"
>>> f.readline()
''
>>> f.close()
```

READING FROM FILES (CONT'D)

- To free up the resources
  - f.close()

```
>>> f = open("somefile.txt", 'r')
>>> f.readline()
"Here's line in the file! \n"
>>> f.close()
```

- With statement
  - Designed to provide clean syntax and exception handling
  - File objects automatically close when they go out of scope

```
with open("text.txt", "r") as filename:
    for line in txt:
        print line
```

```
with open("out.txt", "r") as filename, open ("in.txt","w") as another_filename:
```

STANDARD FILE OBJECTS

- Like cin, cout, and cerr in C++, Python has standard file objects for input, output, and error in the sys module.

```
import sys
for line in sys.stdin:
    print (line)
```

- You can also receive command line arguments from sys.argv[ ]
  - sys.argv[0] is script name

```
for arg in sys.argv:
    print arg
```

>>> python io.py arg1 arg2 arg3

WRITING TO FILES

- f.write(*str*)
  - Writes the string argument *str* to the file object and returns None

```
>>> f = open("filename.txt", 'w')
>>> f.write("Heres a string that ends with " + str(2017))
```

- To update and read the updated file

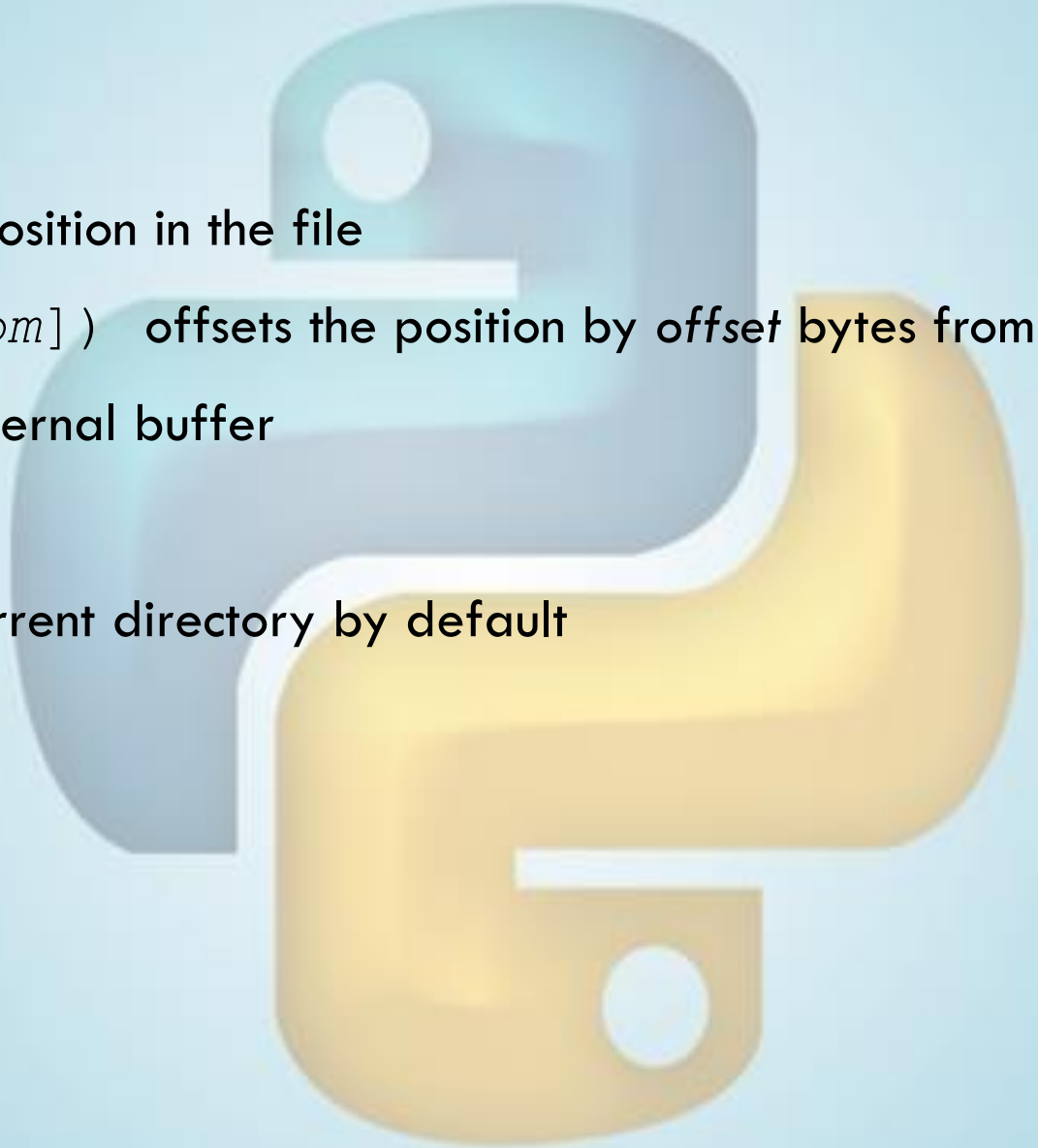
```
>>> f = open("filename.txt", 'r+')
>>> f.write("Heres a string that ends with " + str(2017))
>>> f.readlines()
```

BUILT-IN METHODS

>>> f.open(“file.txt”,”r+”)

- `f.tell()` gives current position in the file
- `f.seek(offset[, from])` offsets the position by *offset* bytes from *from* position
- `f.flush()` flushes the internal buffer

Python looks for files in the current directory by default



MODIFYING FILES AND DIRECTORIES

- `import os`
- `os.rename(current_name, new_name)` renames the file *current_name* to *new_name*.
- `os.remove(filename)` deletes an existing file named filename
- `os.mkdir(newdirname)` creates a directory called *newdirname*
- `os.chdir(newcwd)` changes the cwd to *newcwd*
- `os.getcwd()` returns the current working directory
- `os.rmdir(dirname)` deletes the empty directory *dirname*

EXCEPTIONS

- Errors that are encountered during the execution of a Python program are *exceptions*.

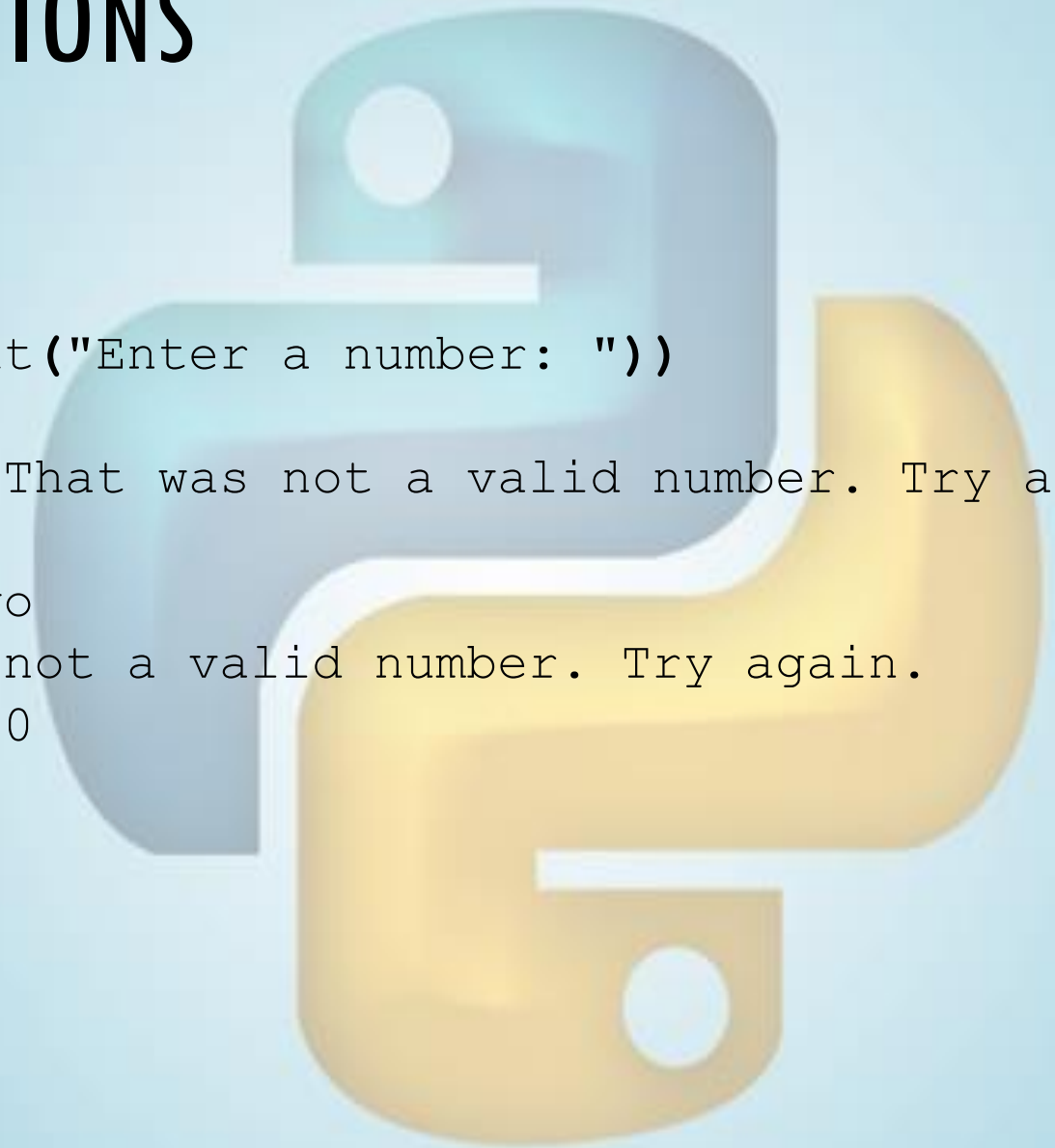
```
>>> print(hi)
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
NameError: name 'hi' is not defined

>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: cannot concatenate 'str' and 'int' objects
```

HANDLING EXCEPTIONS

```
while True:
    try:
        x = int(raw_input("Enter a number: "))
    except ValueError:
        print("Ooops !! That was not a valid number. Try again.") ...

>>> Enter a number: two
>>> Ooops !! That was not a valid number. Try again.
>>> Enter a number: 100
```



HANDLING EXCEPTIONS

- If there is no except block that matches the exception type, then the exception is unhandled and execution stops.

```
while True:
    try:
        x = int(raw_input("Enter a number: "))
    except ValueError:
        print("Ooops !! That was not a valid number. Try again.") ...
```

```
>>> Enter a number: 3/0
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
  File "<string>", line 1, in <module>
ZeroDivisionError: integer division or modulo by zero
```

HANDLING EXCEPTIONS

- The try/except clause options are as follows:

Clause form	**Interpretation**
`except:`	Catch all (or all other) exception types
`except name:`	Catch a specific exception only
`except name as value:`	Catch the listed exception and its instance
`except (name1, name2):`	Catch any of the listed exceptions
`except (name1, name2) as value:`	Catch any of the listed exceptions and its instance
`else:`	Run if no exception is raised
`finally:`	Always perform this block

HANDLING EXCEPTIONS

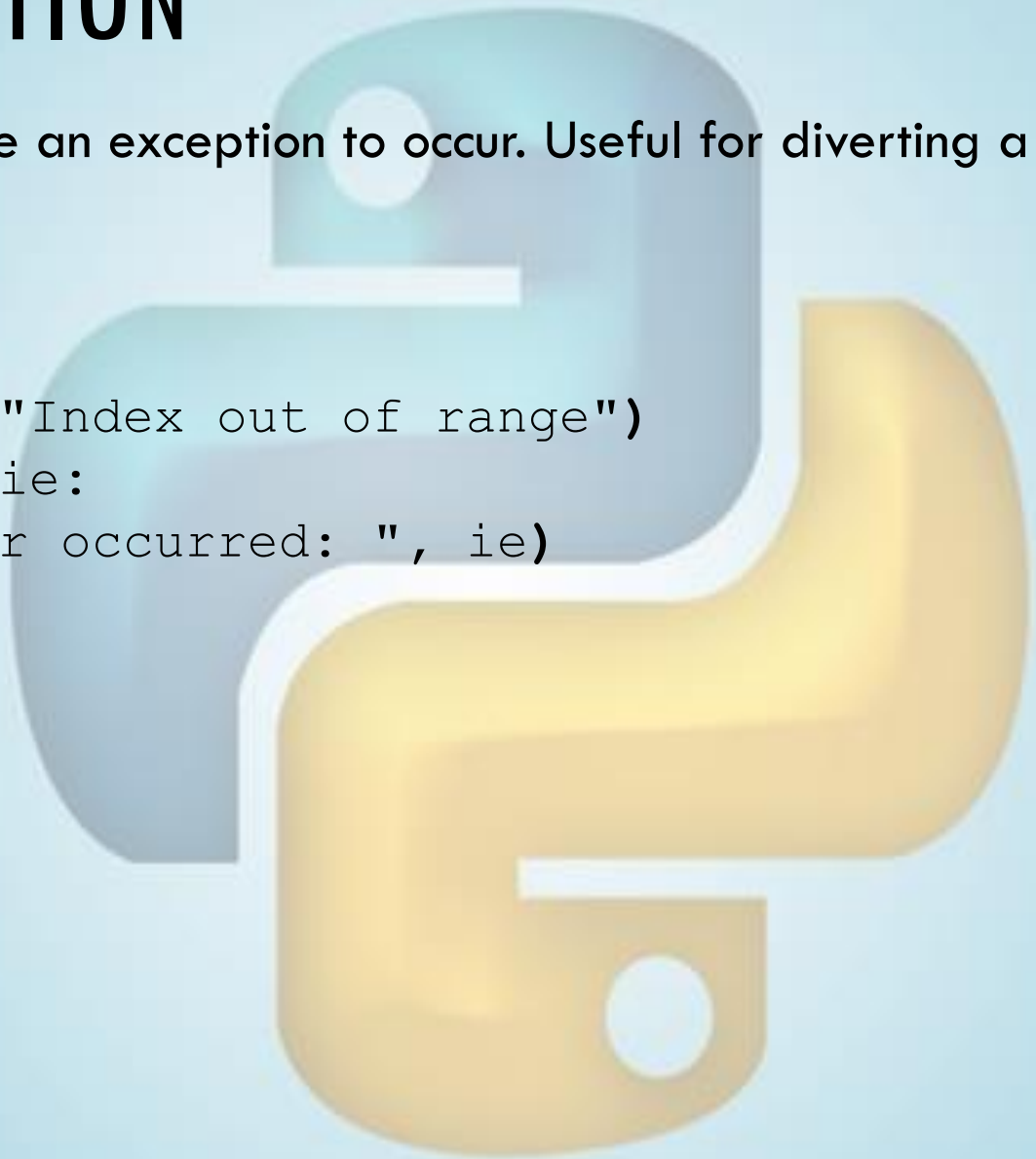
There are a number of ways to form a try/except block.

```
>>> while True:
...     try:
...         x = int(raw_input("Enter a number: "))
...     except ValueError:
...         print("Ooops !! That was not a valid number. Try again.")
...     except (TypeError, IOError) as e:
...         print(e)
...     else:
...         print("No errors encountered!")
...     finally:
...         print("We may or may not have encountered errors…")
...
```

RAISING AN EXCEPTION

- Use the raise statement to force an exception to occur. Useful for diverting a program or for raising custom exceptions

```
try:
    raise IndexError("Index out of range")
except IndexError as ie:
    print("Index Error occurred: ", ie)
```



CREATING AN EXCEPTION

- Make your own exception by creating a new exception class derived from the *Exception* class (we will be covering classes soon).

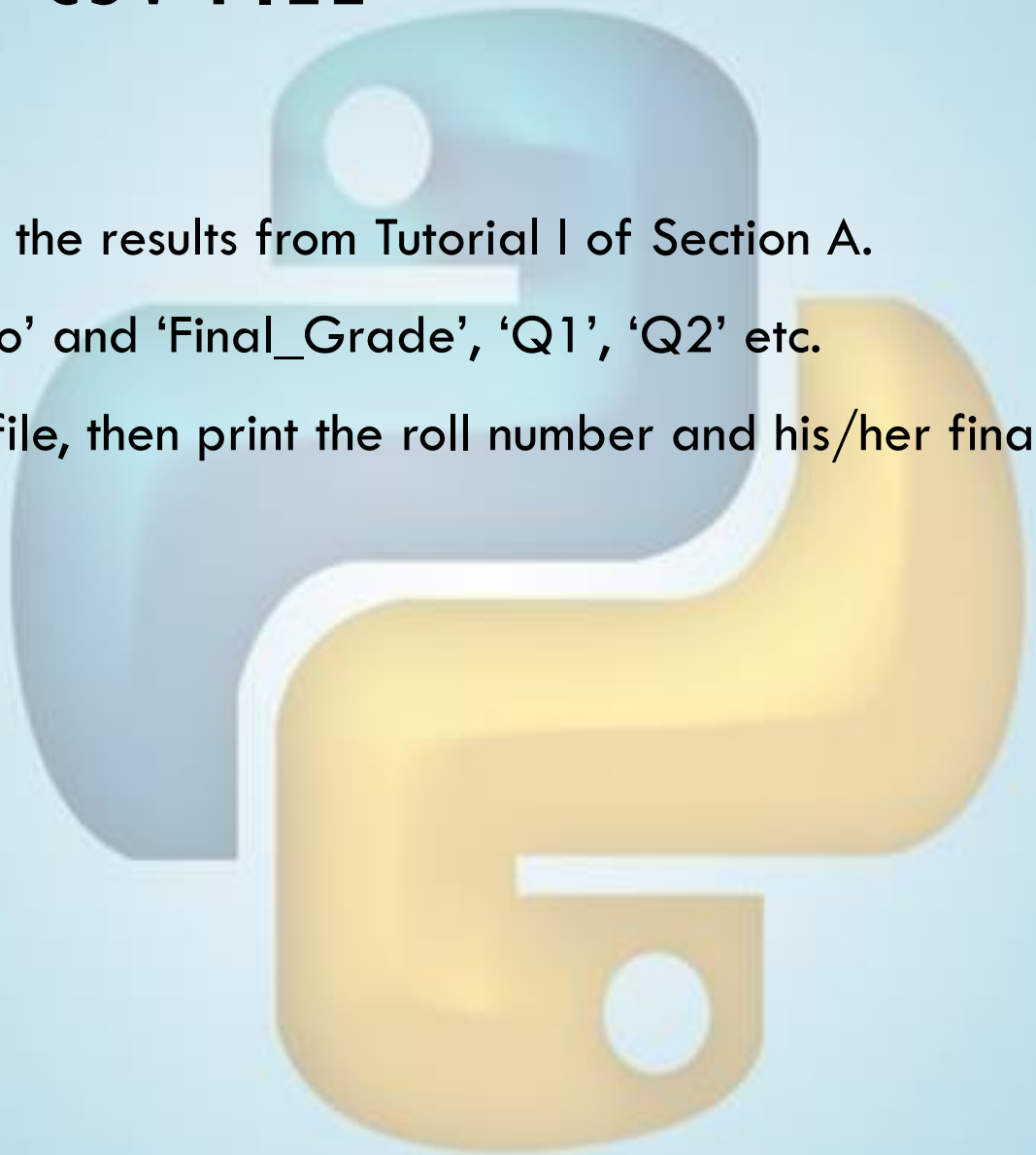
```
class MyError(Exception):
      def __init__(self, value):
          self.value = value
    def __str__(self):
       return repr(self.value)

try:
   raise MyError(2*2)
except MyError as e:
   print 'My exception occurred, value:', e
```

```
>>> My exception occurred, value: 4
```

EXAMPLE: PARSING CSV FILE

- **Tutorial I grades:**
- .csv file given on LMS, contains the results from Tutorial I of Section A.
- The columns labeled as ‘RollNo’ and ‘Final_Grade’, ‘Q1’, ‘Q2’ etc.
- Write a program to read the file, then print the roll number and his/her final grade



MKPT-12345 7

MKPT-12222 4

MKPT-12005 6